X-VLA: Soft-Prompted Transformer as Scalable Cross-Embodiment Vision-Language-Action Model

Jinliang Zheng, Jianxiong Li, Zhihao Wang, Dongxiu Liu, Xirui Kang, Yuchun Feng,
Yinan Zheng, Jiayin Zou, Yilun Chen, Jia Zeng, Ya-Qin Zhang, Jiangmiao Pang,
Jingjing Liu, Tai Wang, Xianyuan Zhan (2025.08.11, ICLR 2026)

Contents

01 VLA 03p

02 Limitation of prior works 04p

03 X-VLA architecture 11p

04 Data processing & Learning recipe 24p

05 Experiments 30p

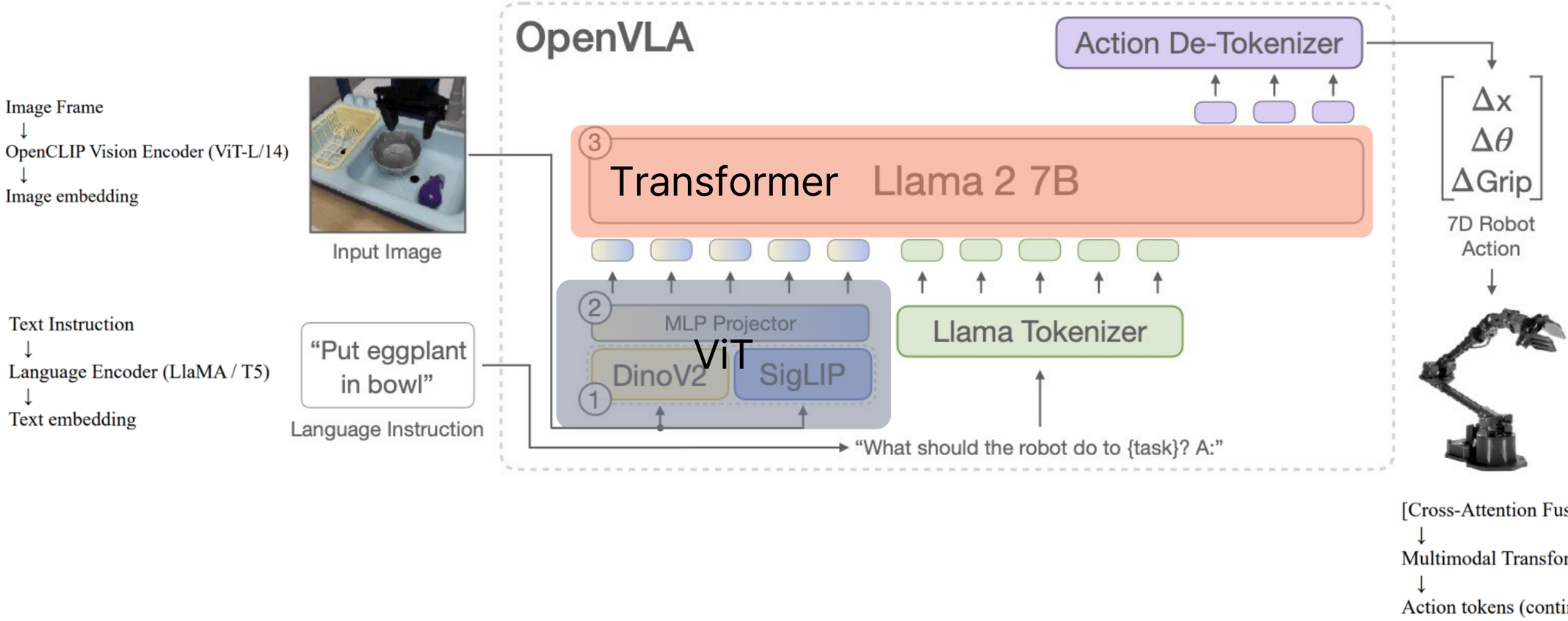
06 Conclusion 34p

VLA

OpenVLA:
An Open-Source Vision-Language-Action Model

Moo Jin Kim[*,1] Karl Pertsch[*,1,2] Siddharth Karamcheti[*,1,3]
Ted Xiao[4] Ashwin Balakrishna[3] Suraj Nair[3] Rafael Rafailov[1] Ethan Foster[1] Grace Lam
Pannag Sanketi[4] Quan Vuong[5,†] Thomas Kollar[3] Benjamin Burchfiel[3] Russ Tedrake[3,6] Dorsa Sadigh[1]
Sergey Levine[2] Percy Liang[1] Chelsea Finn[1]

https://openvla.github.io



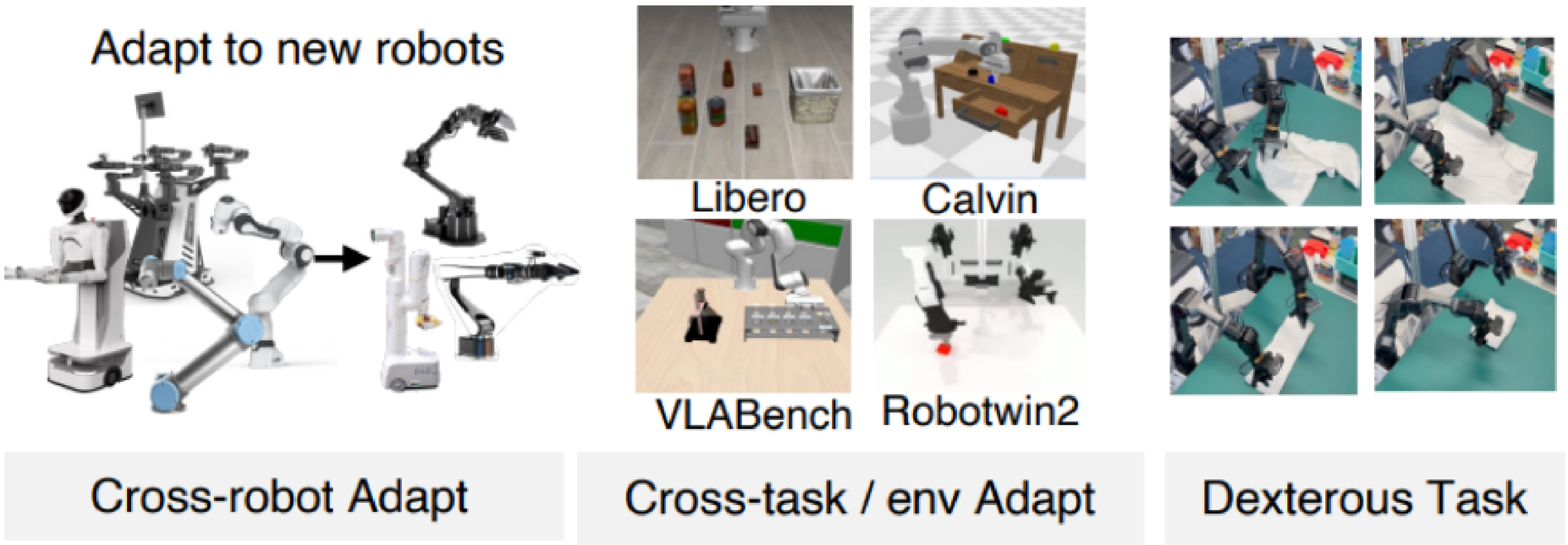
Limitation of prior works

VLA의 목표:

범용 VLA 모델 = 로봇이 다양한 하드웨어·환경에서 동작하도록 해야함

문제점:

- 기존 VLA는 하드웨어 차이와 데이터 불균형에 취약
  - 논문에서는 이를 '이질성(heterogeneity)' 이라고 표현함 (ex. action space, 카메라 설정, 시각적 도메인, 작업 분포 등)



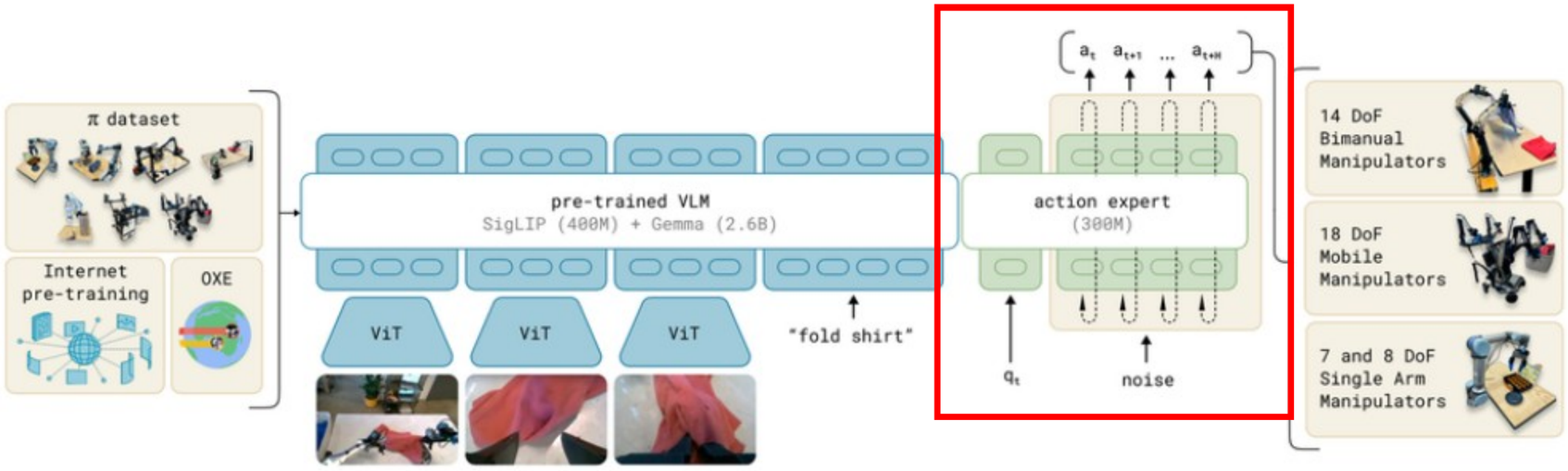
Limitation of prior works

VLA의 목표:

범용 VLA 모델 = 로봇이 다양한 하드웨어·환경에서 동작하도록 해야함

문제점:

- 기존 VLA는 하드웨어 차이와 데이터 불균형에 취약
- 새로운 Task 혹은 Robot 적용 시, **action expert 전체 파라미터** 파인튜닝



ex) OpenVLA

Limitation of prior works

VLA의 목표:

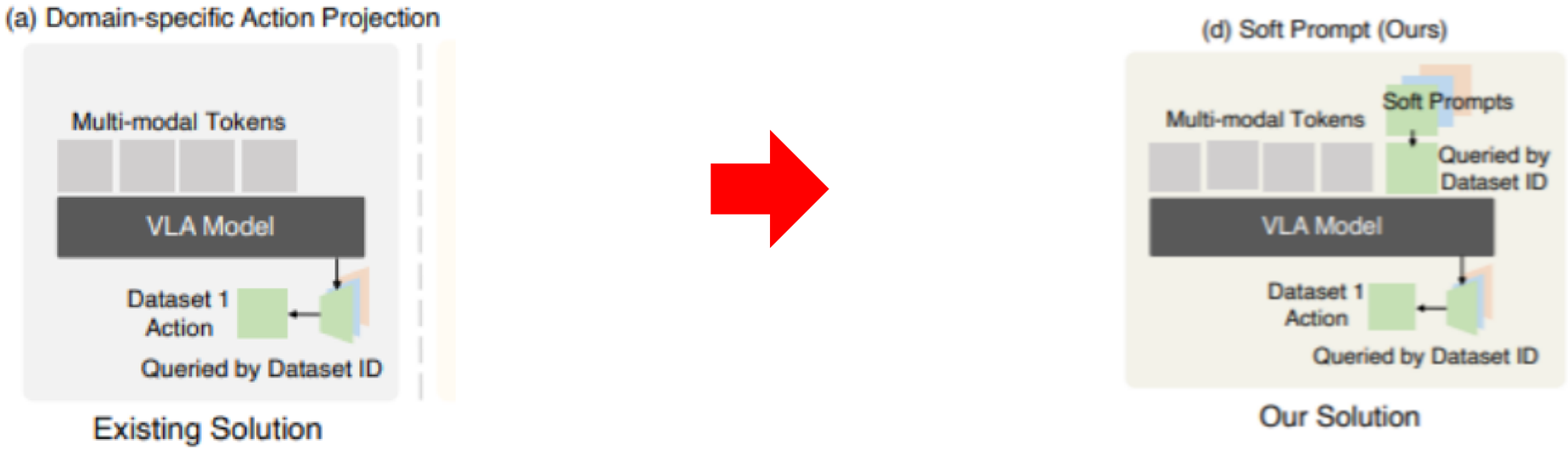
범용 VLA 모델 = 로봇이 다양한 하드웨어·환경에서 동작하도록 해야함

문제점:

- 기존 VLA는 하드웨어 차이와 데이터 불균형에 취약
- 새로운 Task 혹은 Robot 적용 시, **action expert 전체 파라미터** 파인튜닝
  - **Catastrophic Forgetting**: 기존에 학습 Vision-Language 관계**(범용 지식)**와 **이전 Task** 에 대한 행동 패턴이 점진적으로 사라짐
  - **Data Deficiency**: **수천~수만 개**의 데이터 데모 필요

Solution

- (1) 하드웨어 플랫폼 간의 불일치 → [Soft Prompt]
- (2) 표준화된 데이터 수집 프로토콜의 부재 → [Large-scale Pretraining & Soft Prompt]

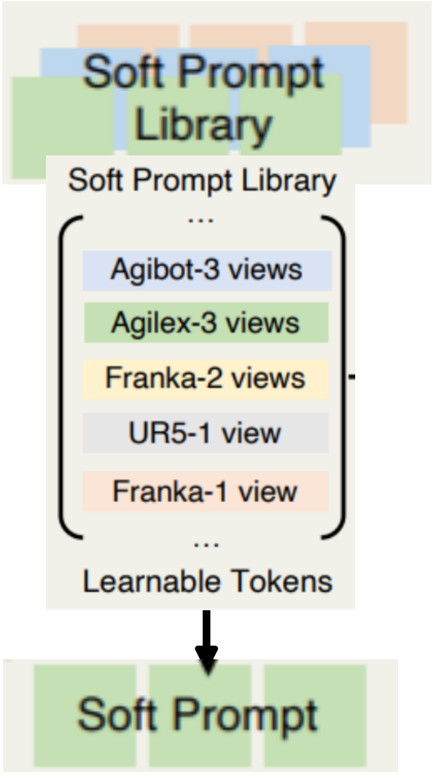


- **Backbone(공통 지식)**: 수많은 환경 데이터를 사전 학습하여 "물체를 어떻게 다뤄야 하는가"에 대한 보편적 물리 지식 학습
- **Soft Prompt(특화 지식)**: 표준 정규분포에서 랜덤 초기화된 벡터들의 집합 → 로봇이나 환경을 구분

ex) 로봇당 8~16개의 토큰 사용

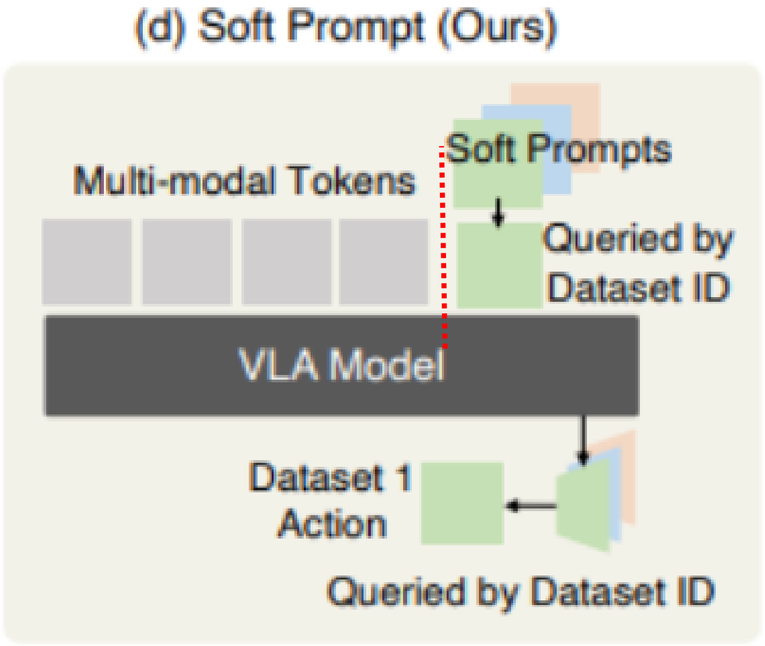
Soft Prompt

- **Meta Learning**
  - 소수의 예제(Few-shot)로 **핵심 특징(Core Knowledge)**만 빠르게 추출하여 새로운 작업에 빠르게 적응하는 능력을 학습
    - 핵심 특징 추출: 각 로봇(데이터 소스)을 하나의 '작업(Task)'으로 보고, 로봇별 특징을 극소량의 파라미터(프롬프트 임베딩)에 저장
    - 새로운 로봇이 등장해도 Backbone(공통 지식)은 고정한 채, 새로운 Soft Prompt만 학습(1% 미만의 파라미터만 튜닝)하면 즉시 적응 가능



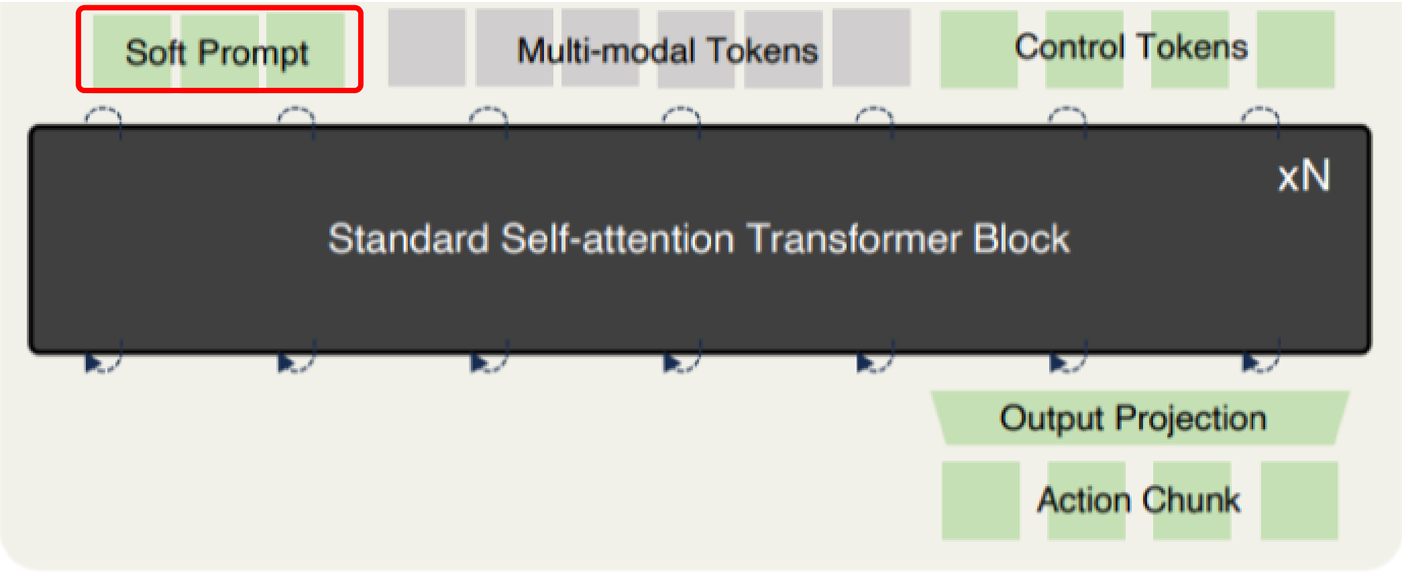
Soft Prompt

- **다중 작업 학습(multi-task learning)**
  - 하나의 모델이 다양한 task 간의 간섭(Interference)을 최소화하면서 공통된 지식 공유
  - 볼로봇, Franka, UR5 등 서로 다른 로봇 데이터를 **"하나의 모델로 동시에 학습"**하되, 각 로봇의 고유 특성이 공통 지식(하나의 ViT/Transformer Backbone)을 '**오염**'시키지 않도록 분리된 Prompt 공간에 격리
  - 다중 작업 학습의 '간섭 방지' 전략을 "공통 트랜스포머 + 작업별 Soft Prompt" 구조로 구현

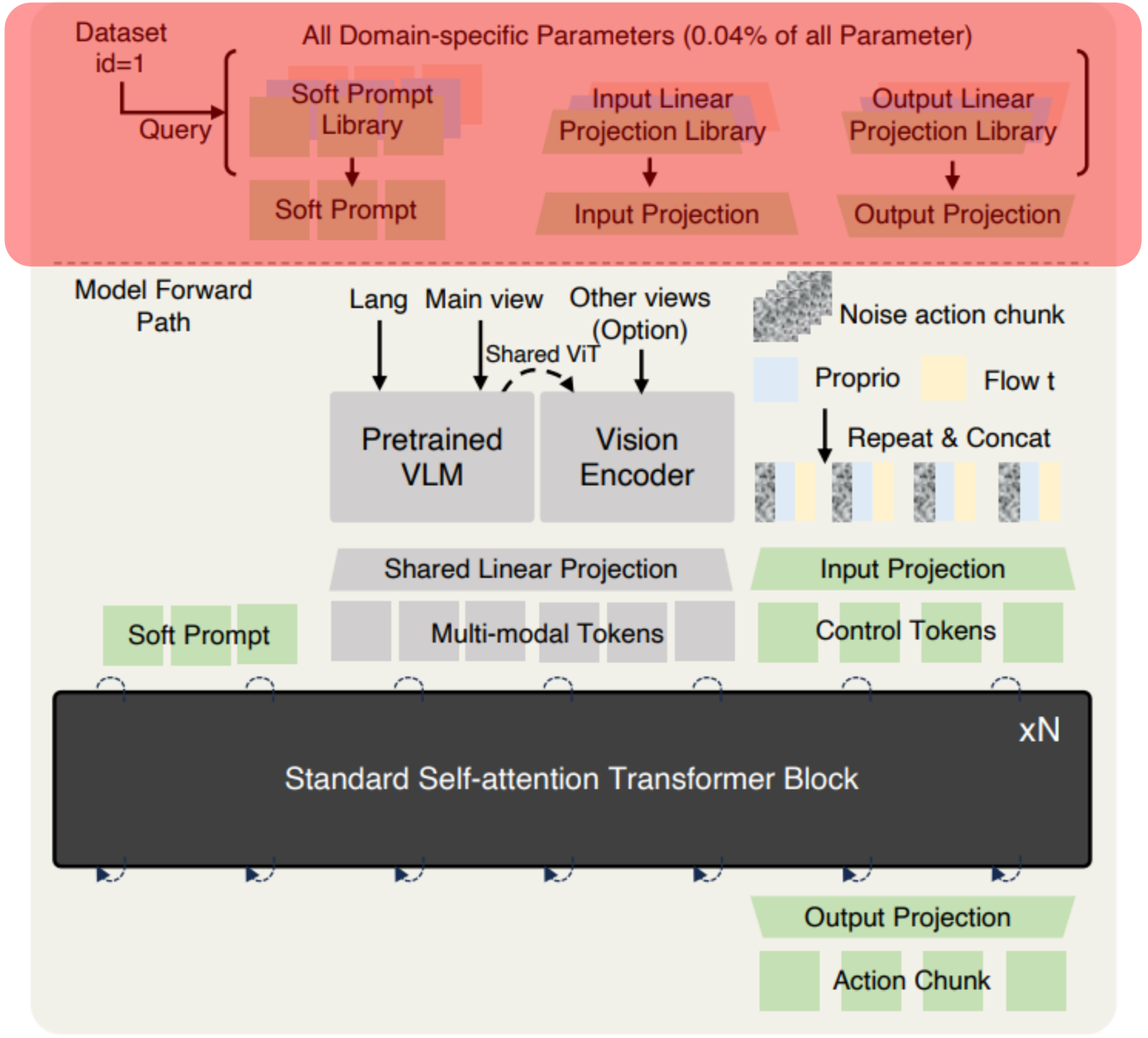


Soft Prompt

- Prompt Learning
  - 대규모 사전 학습 모델(Pre-trained Models)의 지식을 활용하면서도, 새로운 작업(Task)에 적응하기 위해 모델 전체를 변경하지 않고 입력(Input)의 일부를 학습 가능한 파라미터로 바꾸는 기술



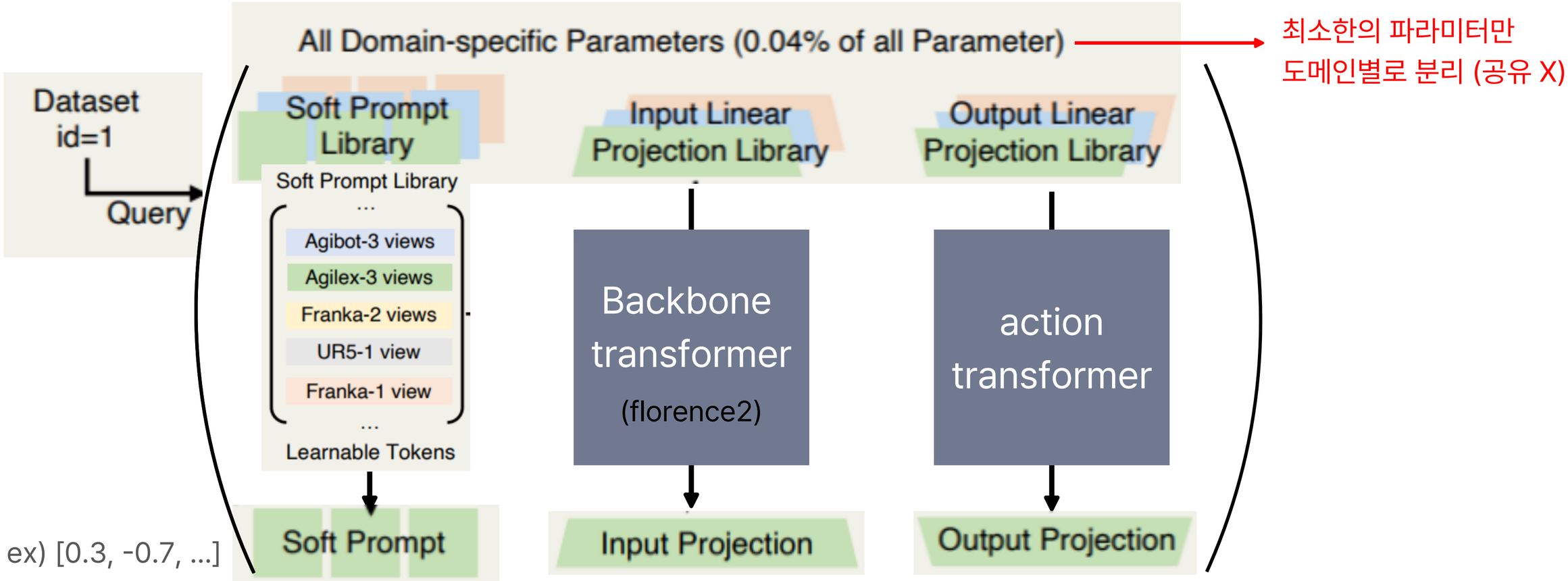
X-VLA



1. Input selection

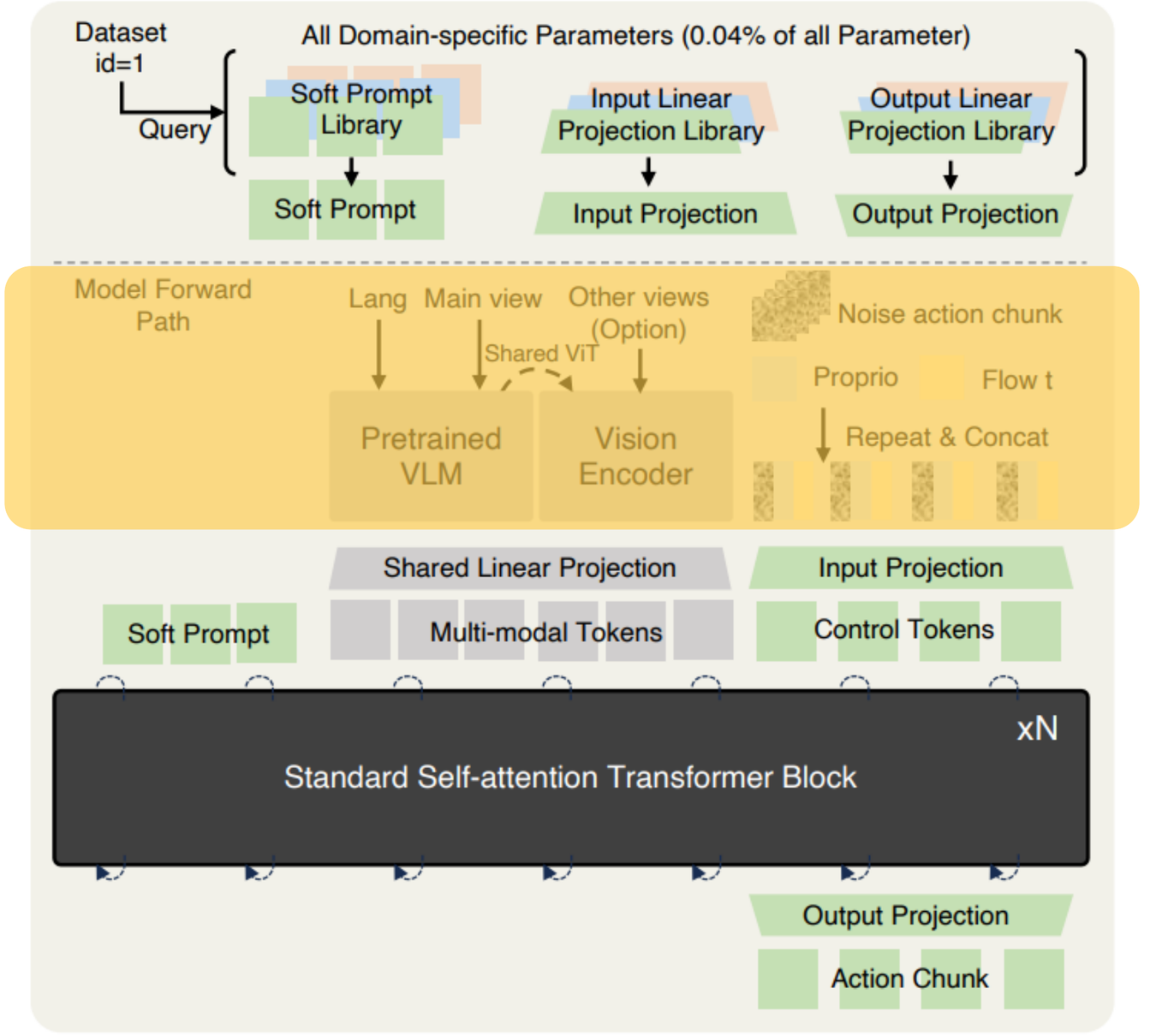
X-VLA

1. Input selection



⇒ Dataset ID 에 맞는 "adapter" 세트

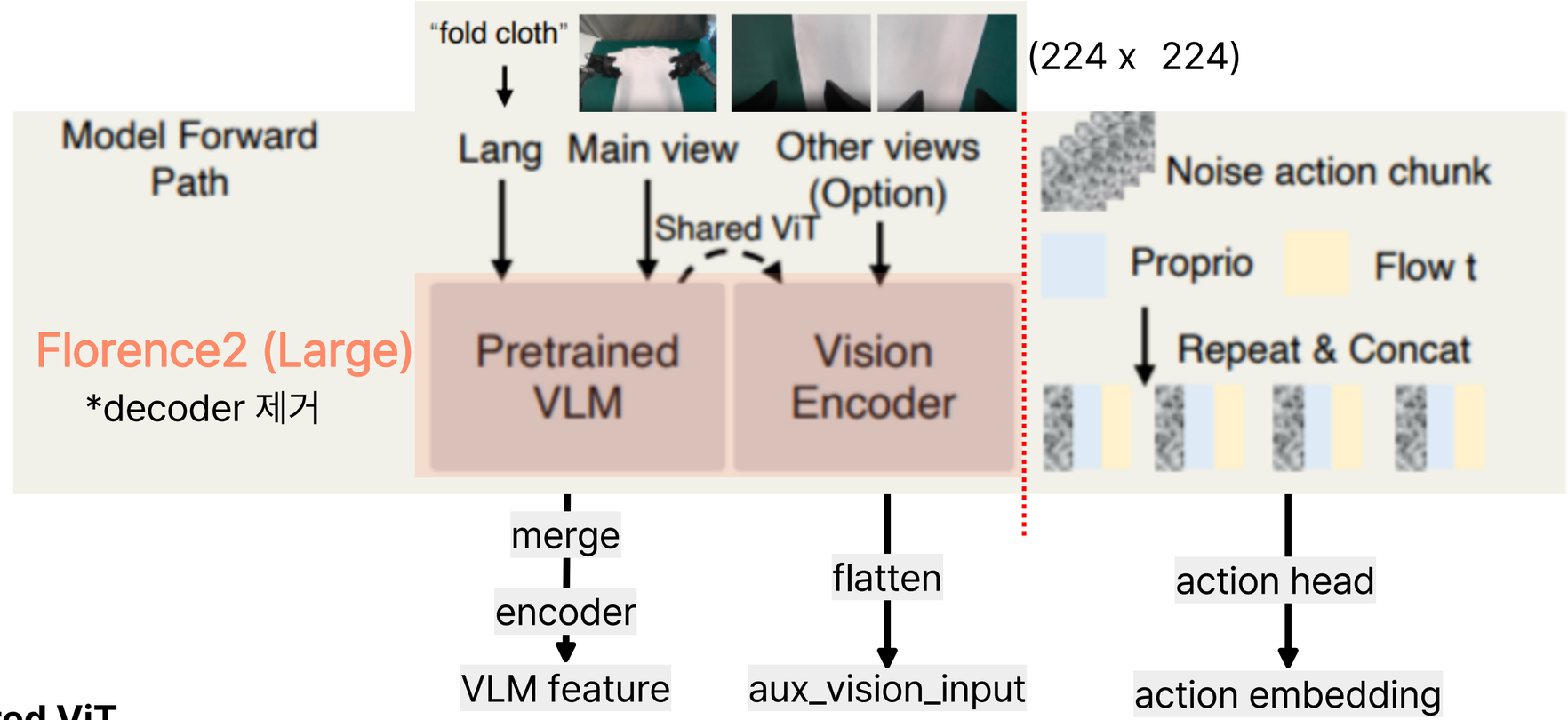
X-VLA



2. Feature extraction

X-VLA

2. Feature extraction



Proprioception: 로봇 상태
(관절 각도, 그리퍼 상태 등)

Flow t: diffusion 시간
(0~1 사이값을 1024차원으로 벡터화)
0: 데이터
1: 완전한 노이즈

***Shared ViT**

- **Main view / Other view 입력 이미지를 따로 받는 이유**
  - **Pre-trained VLM 보존**
    - **1. Multi-modal alignment 유리**
    - **2. zero-shot / Generalization + Castastrophic Forgetting 방지**
    - **3. 데이터 효율 (로봇 view 관련 데이터 상대적 부족)**

Flow Matching

가우시안 분포 N~(0, I)에서 무작위로 추출된 '노이즈' X_1입력



x_1 (노이즈)
randn(B, T, D)

t
i/steps (1→0)
ex) 1.0, 0.9⋯0.1

x_t 혼합
$x_1 \cdot t$ + action·(1-t)
t=1→순수노이즈

Transformer
x_t + proprio
+ VLM 특징
→ 행동 예측

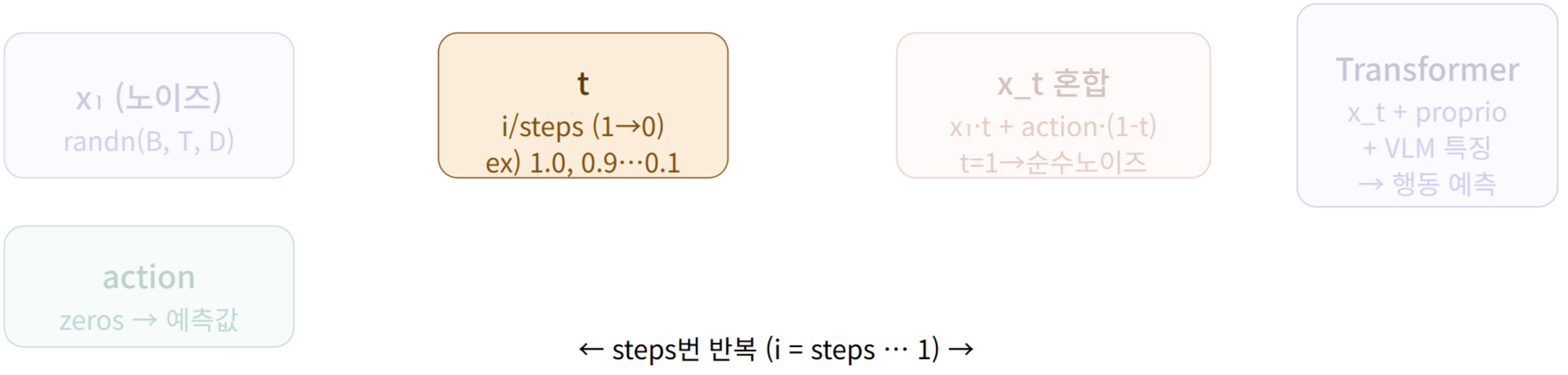
action
zeros → 예측값

← steps번 반복 (i = steps ⋯ 1) →

초기화: 노이즈 x_1 은 순수한 가우시안 랜덤 텐서로 시작합니다. action은 0으로 초기화됩니다. 이 둘이 루프에서 점점 혼합되며 실제 행동으로 수렴합니다.

```
x1 = torch.randn(B, num_actions, D) # 순수 노이즈 action = torch.zeros_like(x1) # 초기 예측 = 0
```

Flow Matching



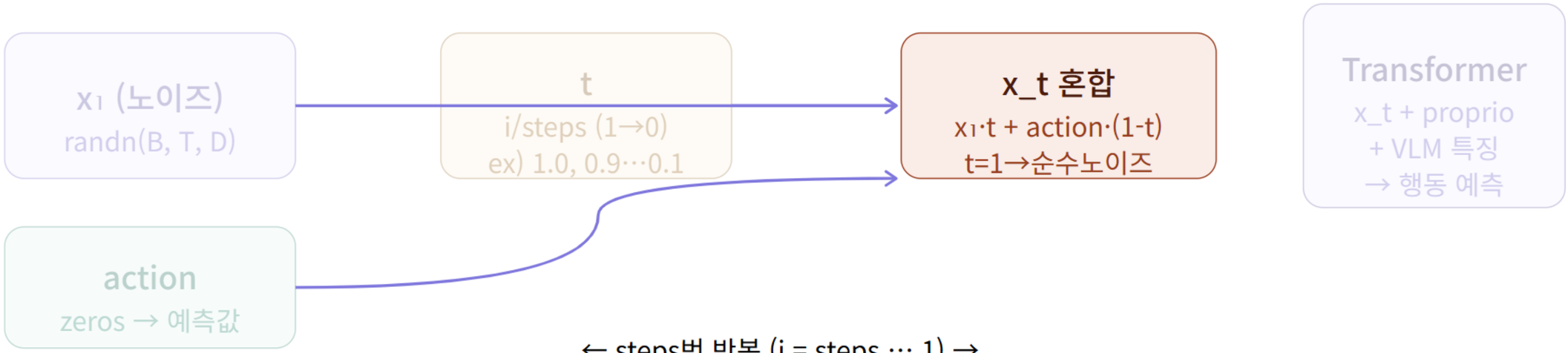
t 계산: 루프 변수 `i`가 `steps`에서 1까지 감소하면서 `t = i/steps`로 계산됩니다. t=1.0이면 완전 노이즈 상태, t→0이면 완전 행동 상태입니다.

```
for i in range(steps, 0, -1): # ex) 10,9,8...1 t = torch.full((B,), i / steps) # 1.0 → 0.1
```

Flow Matching

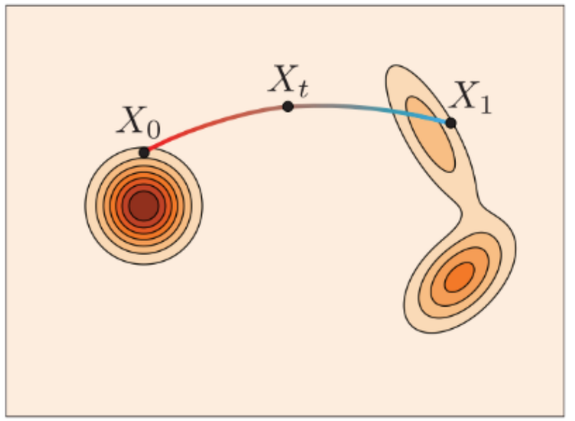
노이즈와 전문가 데이터 분포 간의 선형 매핑(= OT)

At=(1−t)A0+tAgt라는 중간 경로 생성



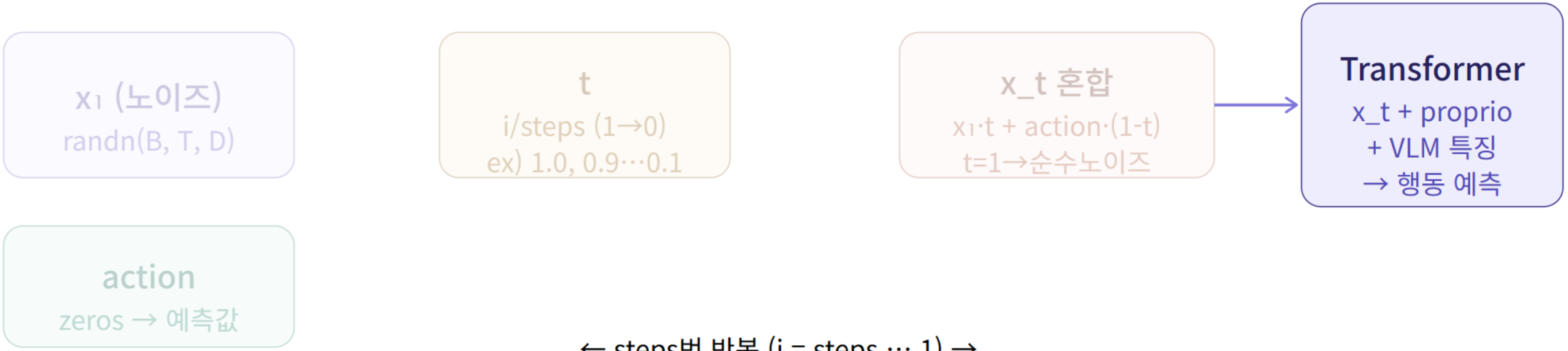
x_t 혼합: x₁ · t + action · (1−t). t가 클수록 노이즈 비중이 높고, t가 작아질수록 이전 스텝의 예측 action이 더 많이 반영됩니다. 선형 보간(Linear Interpolation)입니다.

```
x_t = x1 * t.view(-1,1,1) # 노이즈 비중: t + action * (1 - t).view(-1,1,1) # 예측 비중: 1-t
```



Flow Matching

At를, 관찰값(o) 및 시간(t)과 Concatenate하여 트랜스포머에 입력



Transformer 예측: 혼합된 `x_t`와 `proprio`(관절 상태), VLM 인코더 특징을 입력으로 받아 다음 스텝의 행동을 예측합니다. 이 예측값이 새로운 `action`이 됩니다.

```
action = transformer( action_with_noise=x_t_m, # 혼합된 입력 proprio=proprio_m, # 관절 상태 t=t, **enc # VLM 특징 )
```

Flow Matching

트랜스포머가 입력들을 보고 OT가 정해준 **최적의 속도(Velocity)**를 예측하도록 학습
이 과정을 통해 모델이 벡터 필드를 학습하여 어느 노이즈를 넣더라도 전문가 행동을 출력

x₁ (노이즈)
randn(B, T, D)

t
i/steps (1→0)
ex) 1.0, 0.9⋯0.1

x_t 혼합
x₁·t + action·(1-t)
t=1→순수노이즈

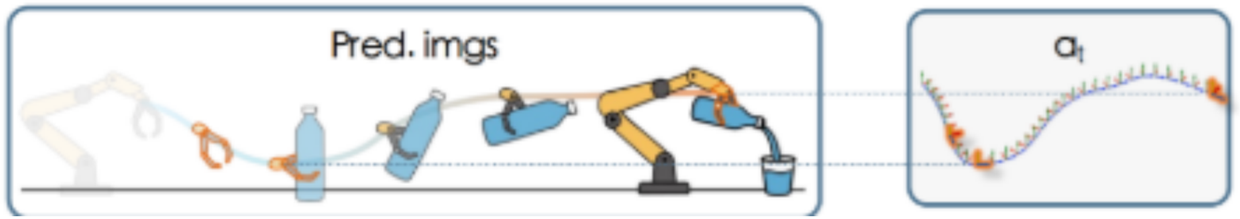
Transformer
x_t + proprio
+ VLM 특징
→ 행동 예측

action
zeros → 예측값

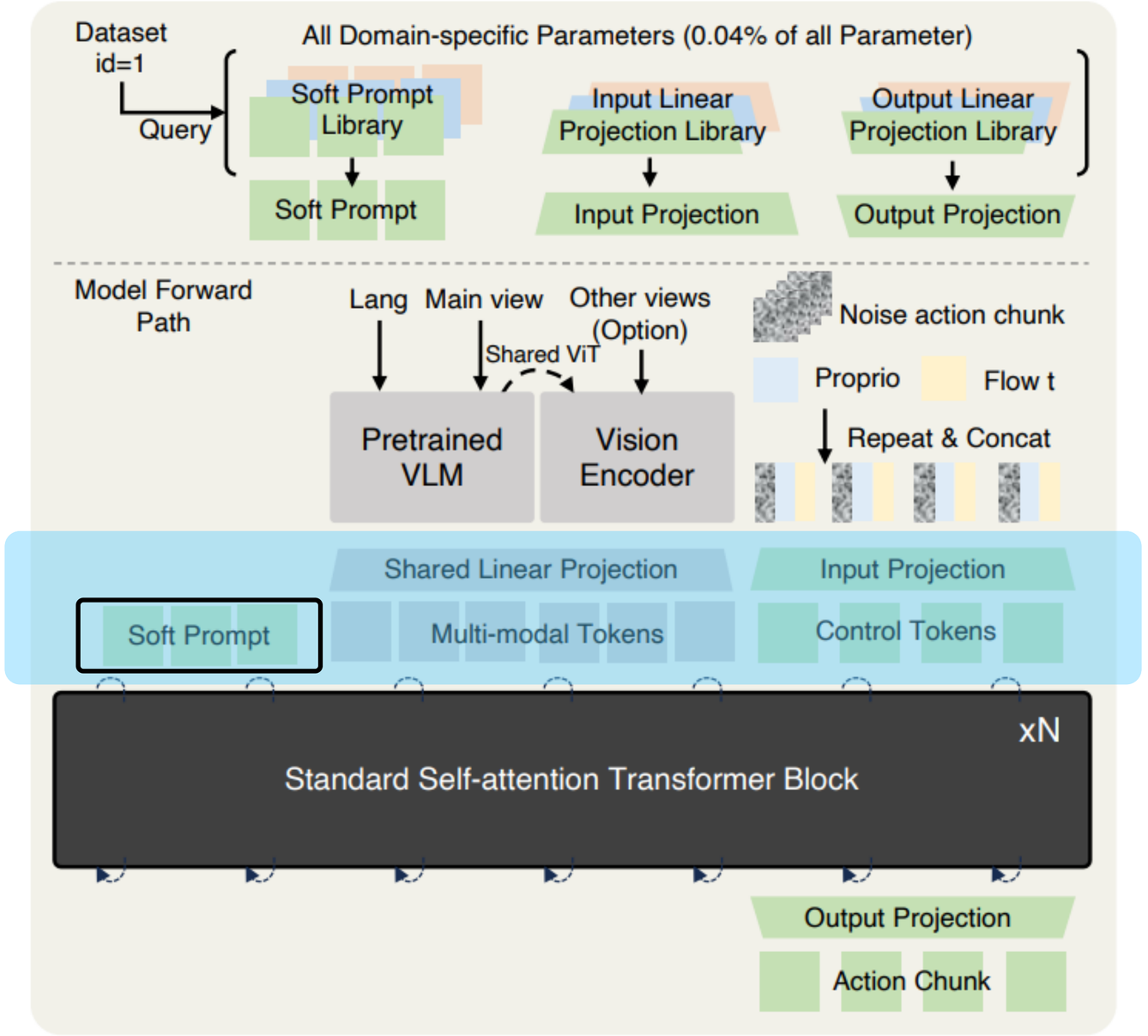
← steps번 반복 (i = steps ⋯ 1) →

반복: 예측된 action이 다음 스텝에서 다시 혼합의 입력으로 들어갑니다. 루프가 끝나면(t→0에 도달) 최종 action에 postprocess(예: gripper sigmoid)를 적용하여 반환합니다.

```
# 루프 종료 후 return action_space.postprocess(action) # 그리퍼 등 후처리
```



X-VLA



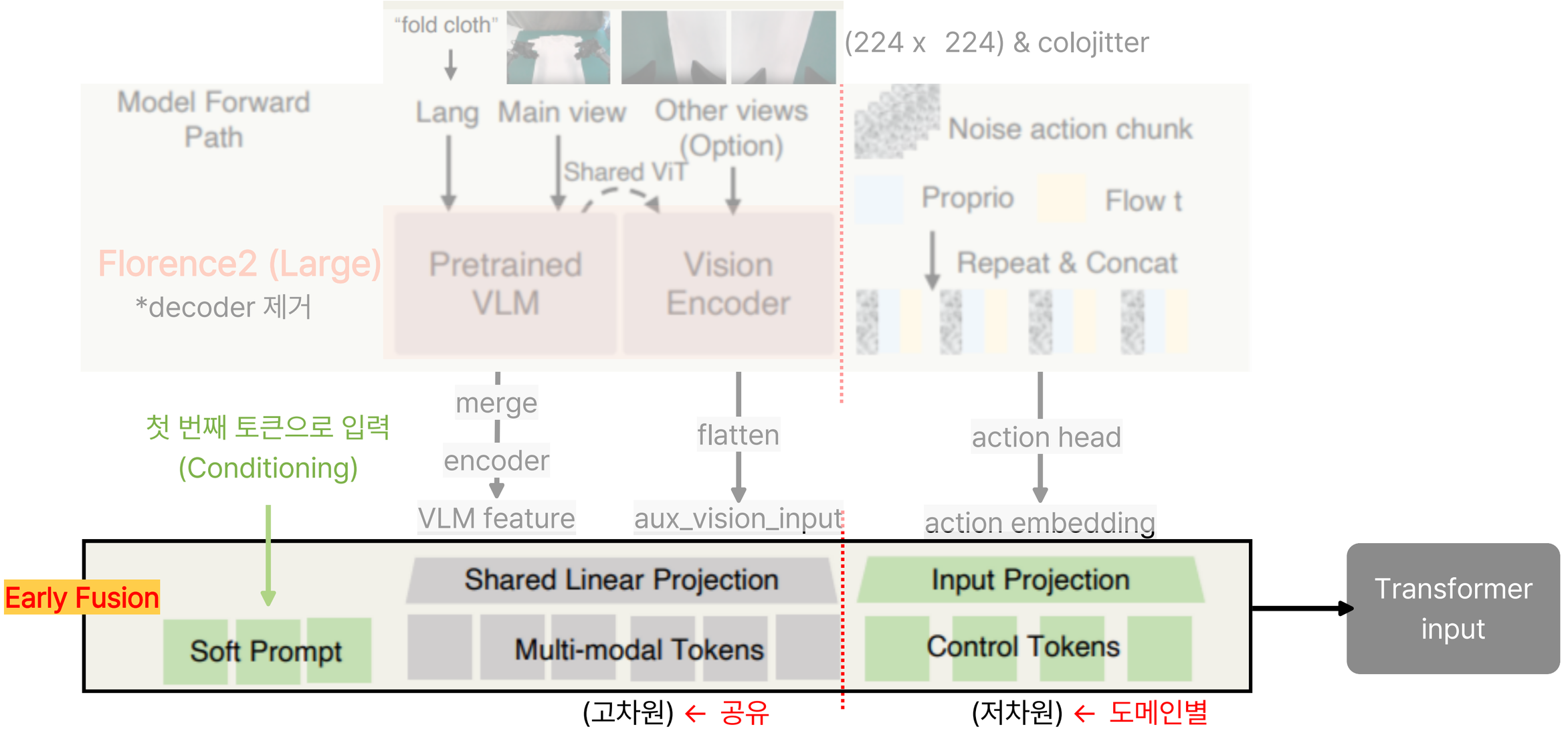
3&4. Tokenization &Conditioning

X-VLA

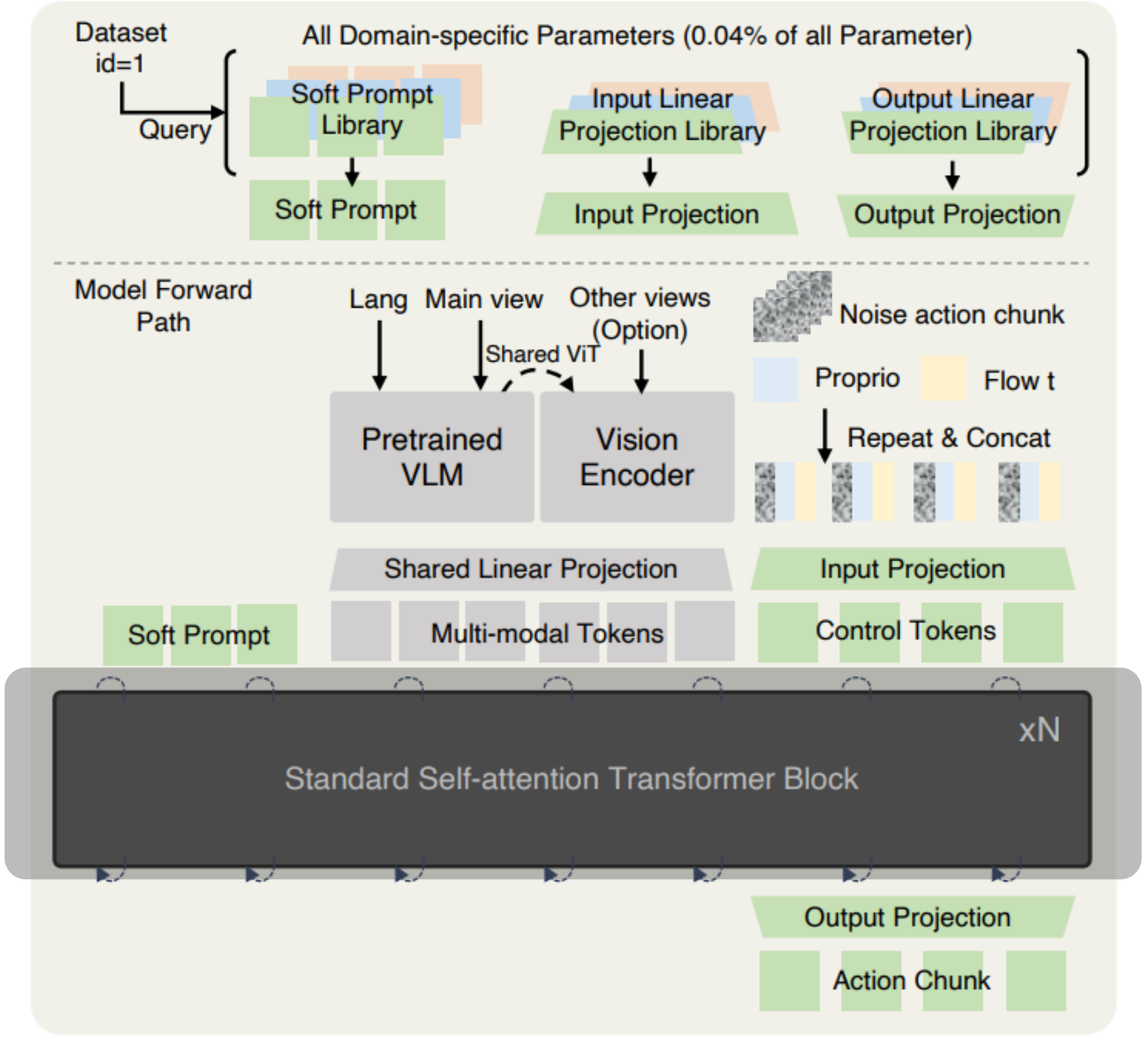
*Multi-modal tokens / Control tokens 분리

- 고차원 토큰과 저차원 토큰을 나누는 이유
  - 학습 안정성

3&4. Tokenization & Conditioning



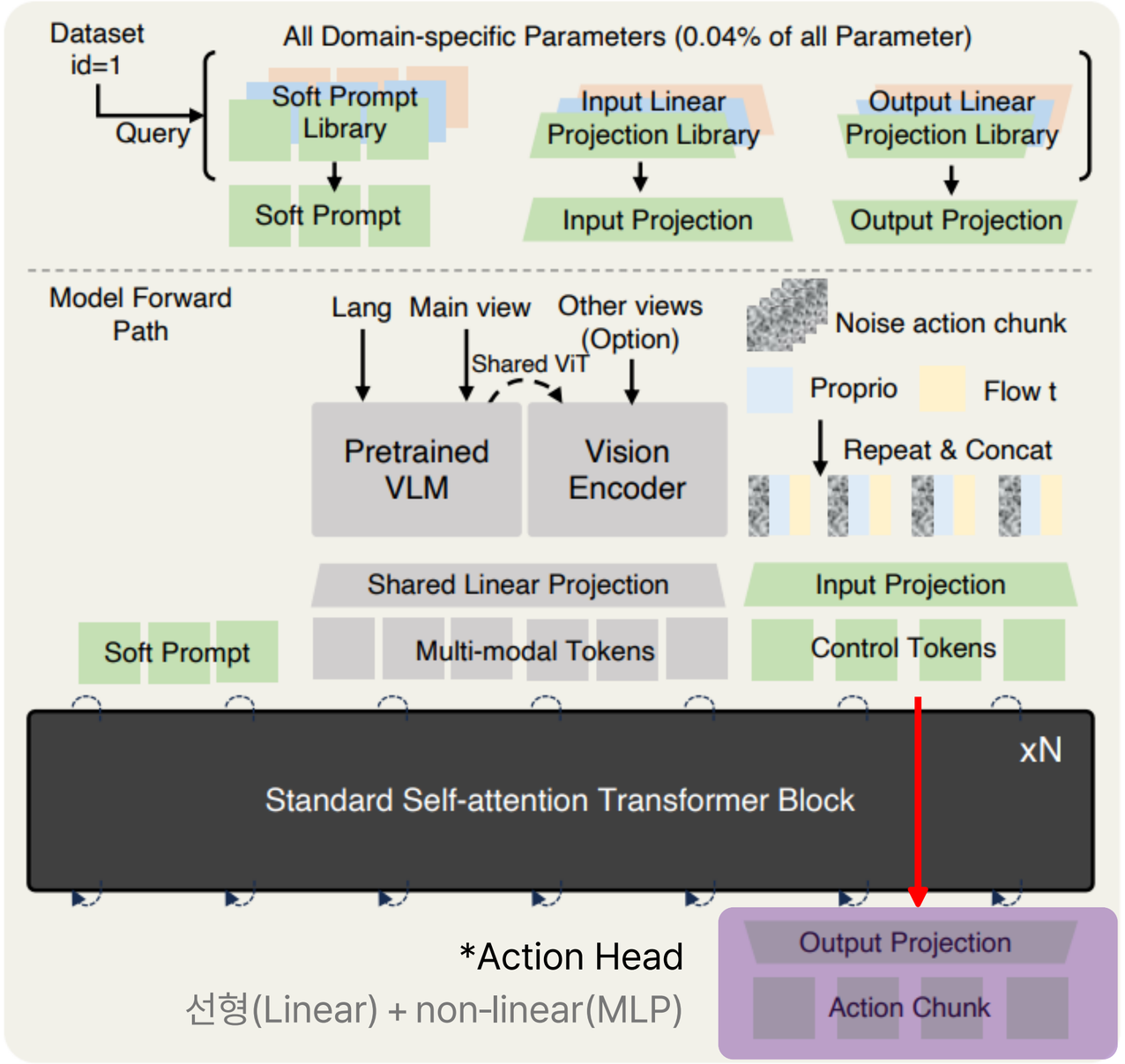
X-VLA



5. Reasoning

- 24 layer(action generation)
- hidden dimention (1024)

X-VLA



6. Output mapping

θ, X, Grip 변환

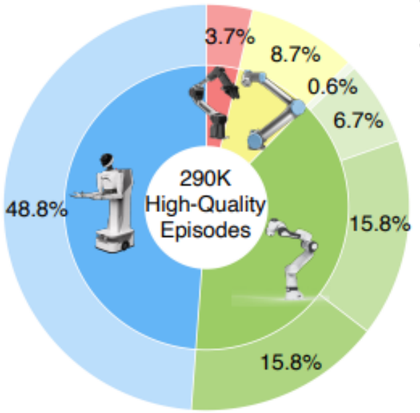
Pre-traning

Data processing & Learning recipe

데이터셋:

- 290K epi, 5 robots, 7 platform
- Shuffling pipline 설계
  - Cross-domain shuffling
  - Cross-trajectory shuffling
  - Weighted sampling 적용

    - 특정 도메인으로의 overfitting 방지
    - 멀티 도메인 generalization 향상



Embodiment	Freq	Source	Camera Setup
AGIBOT(48.8%)	*30Hz*	AGIBOT-Beta	Head/Wrist
Franka(15.8%)	*15Hz*	Droid	Left/Wrist
Franka(15.8%)	*15Hz*	Droid	Right/Wrist
Franka(6.7%)	*30Hz*	RoboMind	Top
Dual-Franka(0.8%)	*30Hz*	RoboMind	Front/Wrist
UR-5(8.7%)	*30Hz*	RoboMind	Top
Agilex(3.7%)	*30Hz*	RoboMind	Front/Wrist

Figure 3 | The recipe for mixed data used in pretraining experiments.

Data source	Num. traj	Sampling weight
AGIBOT	141K	0.4
Droid-Left	45K	0.15
Droid-Left	45K	0.15
RoboMind-Franka	19K	0.1
RoboMind-Dual-Franka	2K	0.03
RoboMind-UR	25K	0.1
RoboMind-Agilex	11K	0.07

Table 8 | Sampling weights for heterogeneous data sources during pretraining.

- AGIBOT: 일상 가사/조립 작업 고품질의 대규모 데이터로, 다중 뷰 카메라 시점 포함
- Droid: 상대적으로 시점이 제한적이며 주파수가 낮아, AGIBOT와 대조됨
- RoboMind: 산업/실험실 환경 4개의 하드웨어 변형으로 구성되어 하드웨어 다양성 제공

Pre-traning

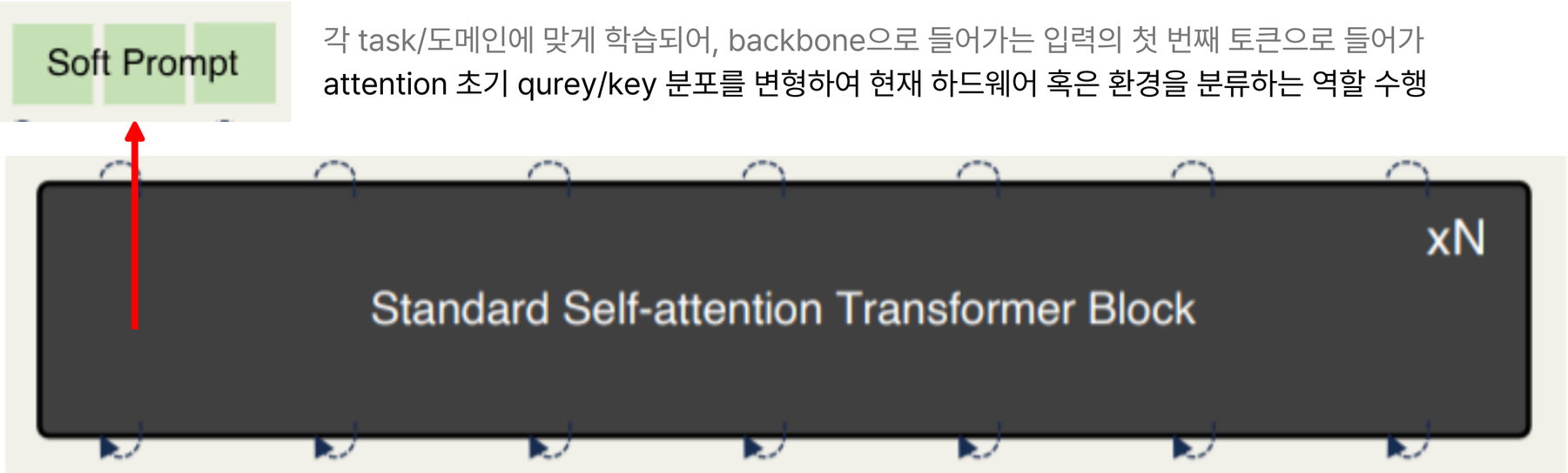
Data processing & Learning recipe

환경:

- NVIDIA A100 GPU (X64)
- Global batch 1024 (4days)
- Optimizer: AdamW (Beta_1: 0.9, Beta2: 0.95)
- Learning Rate: 1 x 10^-4
- Weight decay: 0.01
- Traning steps: 200K iterations
- Image input
  - 224 x 224 (resize)
  - Colorgitter (for generalization)

Backward

backbone(범용 정책)과 Soft-prompt를 동시에 학습



각 task/도메인에 맞게 학습되어, backbone으로 들어가는 입력의 첫 번째 토큰으로 들어가
attention 초기 qurey/key 분포를 변형하여 현재 하드웨어 혹은 환경을 분류하는 역할 수행

(노이즈와 전문가 데이터 의 중간단계)

A_t = (1-t)A_0 + tA

$$\mathcal{L}_{\mathrm{BC}}^{\mathrm{FM}}(\theta) = \mathbb{E}_{t\sim\mathcal{U}(0,1),\,(o,A)\sim\mathcal{D}}\left[\left\|v_\theta(A^t, o, t) - (A - A^0)\right\|^2\right],$$

현재위치(예측값)　　목표위치

시간 t를 0에서 1까지 무작위 추출

데이터셋 D에서 실제관찰(o)과
전문가 행동(A) 샘플을 뽑음

Adaptation

기존에는 새로운 작업(예: 특정 로봇 제어)에 적응시키기 위해
전체 모델 파라미터 Fine-tuning

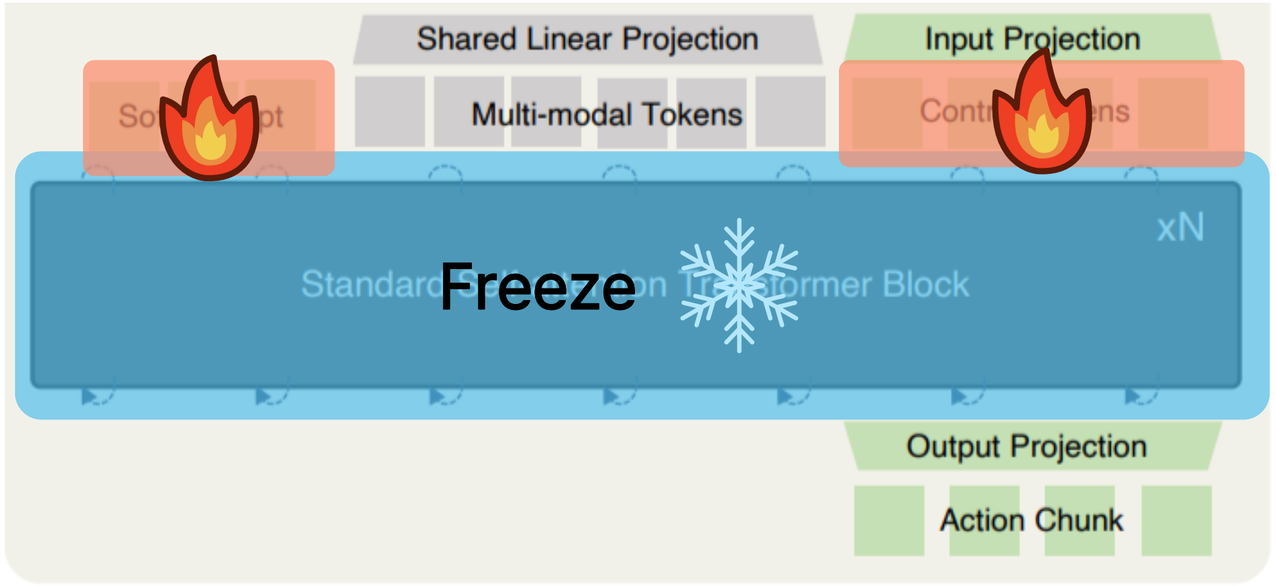
- 문제점 1: 수십억 개의 파라미터를 모두 업데이트해야 해서 계산 비용이 엄청나게 큼.
- 문제점 2: 새로운 데이터로 학습하다 보면 원래 모델이 알고 있던 범용 지식을 망각하게 됨.

Adaptation

2단계 적응 전략 (Two-step procedure)

1. Prompt warm-up (First 1,000 iterations)
- soft prompt + action head만 업데이트
- 나머지 파라미터는 모두 freeze

> pretrained backbone을 최대한 보존하면서 빠르게 도메인에 맞는 "입력/출력 인터페이스"만 조정



Adaptation

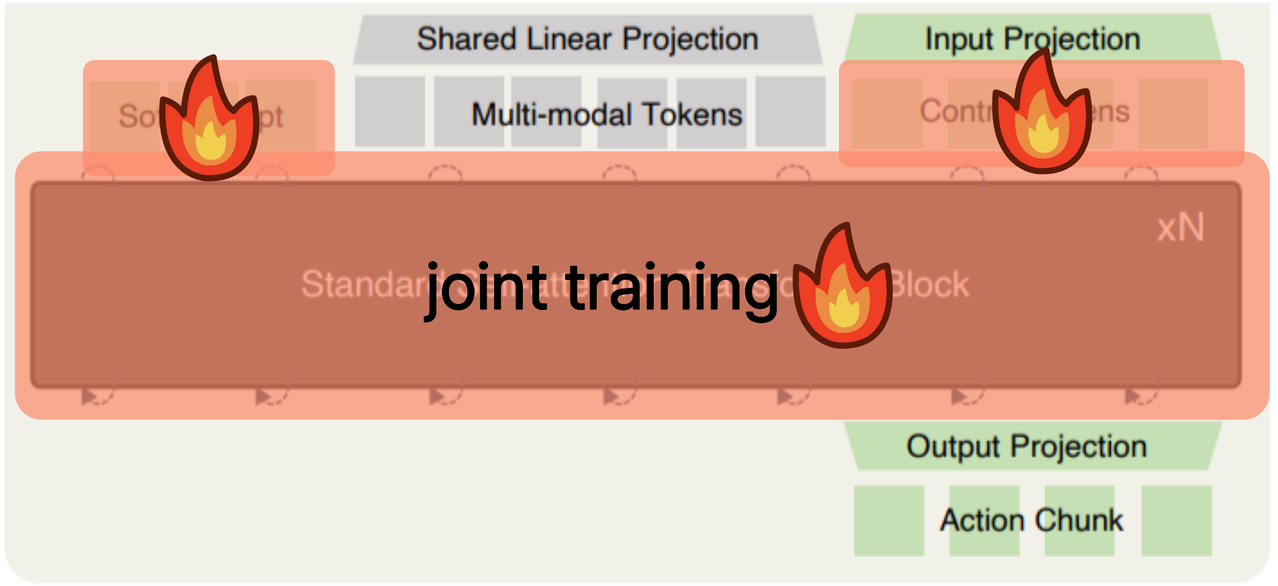
2단계 적응 전략 (Two-step procedure)

2. Joint policy adaptation (다음 1,000 iterations)

- 학습률을 낮은 값에서 점진적으로 원래 값으로 복원
- 이후 전체 파라미터를 함께 학습 (joint training)

> 초기 alignment 이후, 전체 모델을 미세하게 조정

*밴치마크에 난이도에 따라 학습 횟수가 상이함



Experiments

X-VLA 설계 기법 성능 변화 (Table 1)

Type	Improvements	Val Error (PT)	Acc (AD)
Baseline Model (w/o PT)	Florence-base + Standard DiT-base	-	4.1
Pretraining Technique (Section 4.2.1)	+Custom LR (w/o PT)	-	39.6 (+35.5)
	+Heterogeneous PT	0.11	25.0 (-14.6)
Data Processing (Section 4.2.2)	+Action alignment +Intension abstraction +Balanced data sampling	0.077 (-0.033)	50.0 (+25.0)
Architecture Design (Section 4.1)	+Replace DiT with Transformer encoder	0.071 (-0.006)	47.9 (-2.1)
	+Encoding pipeline	0.053 (-0.018)	64.6 (+16.7)
	+Soft-prompt	0.041 (-0.012)	73.8 (+9.2)
	+Scaling up	0.032 (-0.009)	**89.6** (+15.8)
Finetuning Technique (Section 4.2.1)	+Two-step adaptation	0.032	**95.8** (+6.2)

단순히 여러 로봇 데이터를 섞어서 학습할 경우 25% 낮은 성공률을 보임

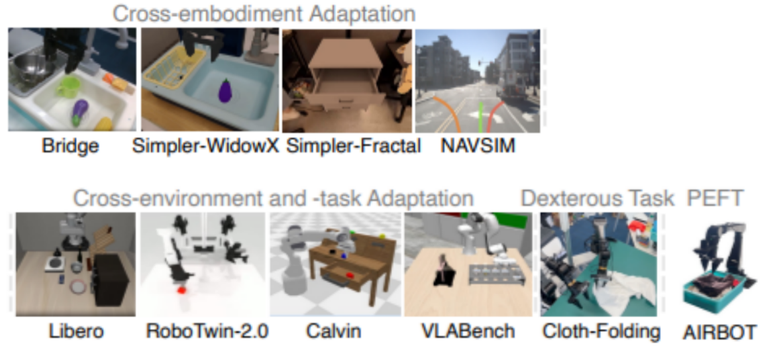
아키텍처 단에서 soft-prompt를 적용했을 때, 성공률이 9%이상 대폭 증가

Experiments

6개의 서로 다른 시뮬레이션 벤치마크 (Table 2)

Table 2 | Comparison of specialize and generalize models on simulation benchmarks

Methods	Size	Simpler			LIBERO					Calvin	RoboTwin-2.0		VLABench	NAVSIM
		VM	VA	WidowX	Spatial	Object	Goal	Long	Avg	ABC → D	Easy	Hard	Avg. PS	PDMS
LBP (Liu et al., 2025a)	0.2B	-	-	-	-	-	-	88.6	-	-	-	-	-	-
MoDE (Reuss et al., 2024)	0.4B	-	-	-	-	-	-	94.0	-	4.01	-	-	-	-
SuSIE (Black et al., 2024b)	1B	-	-	-	-	-	-	76.3	-	2.69	-	-	-	-
GHIL-Glue (Hatch et al., 2025)	1B	-	-	-	-	-	-	-	-	3.69	-	-	-	-
SpatialVLA (Qu et al., 2025)	4B	75.1	70.7	42.7	88.2	89.9	78.6	55.5	78.1	-	-	-	-	-
TraceVLA (Zheng et al., 2024b)	7B	46.2	49.1	-	84.6	85.2	75.1	54.1	74.8	-	-	-	-	-
ThinkAct (Huang et al., 2025)	7B	71.5	65.1	43.8	88.3	91.4	87.1	70.9	84.4	-	-	-	-	-
FPC-VLA (Yang et al., 2025)	7B	78.0	65.8	64.6	86.2	87.0	92.0	82.2	86.9	-	-	-	-	-
MemoryVLA (Shi et al., 2025a)	7B	77.7	72.7	71.9	**98.4**	98.4	96.4	93.4	96.7	-	-	-	-	-
Octo (Octo Model Team et al., 2024)	0.1B	16.8	1.10	23.4	78.9	85.7	84.6	51.1	75.1	-	-	-	-	-
GR-1 (Wu et al., 2023)	0.2B	-	-	-	-	-	-	-	-	3.06	-	-	-	-
Seer (Tian et al., 2025)	0.3B	-	-	-	-	-	-	87.7	-	4.28	-	-	-	-
UniAct (Zheng et al., 2025)	0.5B	-	-	-	77.0	87.0	77.0	70.0	77.8	-	-	-	-	-
RDT (Liu et al., 2025b)	1B	-	-	-	-	-	-	-	-	-	34.5	13.7	-	-
FLOWER (Reuss et al., 2025)	1B	-	-	40.0	97.1	96.7	95.6	93.5	95.7	**4.53**	-	-	-	-
SmolVLA (Shukor et al., 2025)	2B	-	-	-	93.0	94.0	91.0	77.0	88.8	-	-	-	-	-
GR00T-N1 (Bjorck et al., 2025)	3B	45.0	48.4	-	94.4	97.6	93.0	90.6	93.9	-	-	-	39.7	-
π_0 (Black et al., 2024a)	3B	58.8	56.8	27.8	96.8	**98.8**	95.8	85.2	94.1	-	46.4	16.4	37.8	-
π_0+FAST (Pertsch et al., 2025)	3B	61.9	60.5	39.5	96.4	96.8	88.6	60.2	85.5	-	-	-	34.1	-
OpenVLA (Kim et al., 2024)	7B	-	-	8.30	84.7	88.4	79.2	53.7	76.5	-	-	-	-	-
OpenVLA-OFT (Kim et al., 2025)	7B	63.0	54.3	31.3	97.6	98.4	**97.9**	94.5	97.1	-	-	-	-	-
DD-VLA (Liang et al., 2025)	7B	71.2	64.1	49.3	97.2	98.6	97.4	92.0	96.3	-	-	-	-	-
UniVLA (Wang et al., 2025a)	9B	-	-	69.8	95.4	**98.8**	93.6	94.0	95.4	4.41	-	-	-	81.7
Maximum of Existing SOTA	-	78.0	72.7	71.9	98.4	98.8	97.9	94.5	97.1	4.53	46.4	16.4	39.7	81.7
X-VLA (Ours)	0.9B	80.4	75.7	95.8	98.2	98.6	97.8	97.6	98.1	4.43	70.0	39.0	51.1	87.3



Libero(공간/객체/목표/장기 추론)
Calvin(환경 변경)
NAVSIM(자율 주행)

구조(Soft Prompt)가 크기보다 중요

X-VLA (0.9B) > OpenVLA, UniVLA 등(7B~9B)

범용성 입증

Libero, Calvin, NAVSIM 등
서로 다른 환경들에서 모두 SOTA

Experiments

Adaptation Evaluation

- Simple Manipulation (WidowX 로봇):

의미: 가장 기본적이면서도 실제 환경에서 가장 많이 쓰이는 Pick-and-Place 작업

검증 목표: 시뮬레이션에서 학습한 지식이 실제 로봇의 모터와 센서 환경으로 얼마나 잘 전이(Transfer)되는가?

- Dexterous Manipulation (Agilex 로봇):

의미: 양팔(Bi-manual)을 사용하며 정교한 제어가 필요한 작업

검증 목표: 고난도, 정밀 작업(Dexterity)에서도 X-VLA가 동작하는가? (옷 개기 실험)

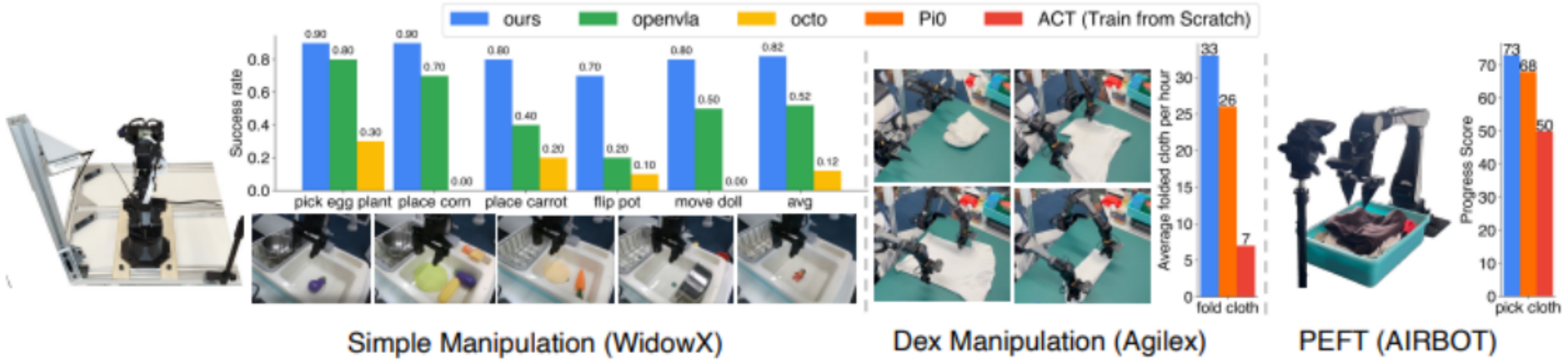
- PEFT Adaptation (AIRBOT):

의미: 학습 단계에서 한 번도 보지 못한(Unseen) 새로운 로봇에 대한 적응 실험

검증 목표: 사전 학습된 지식을 바탕으로 단 200개의 적은 데모(Few-shot)만으로 새로운 하드웨어에 얼마나 빠르게 적응하는가?

Experiments

Real-World Evaluation Results (Figure 7)



첫째, 복잡한 양팔 정밀 조작(Dexterous)이 가능하다

둘째, 사전 학습에 없었던 새로운 로봇(AIRBOT)에도 소수의 데이터만으로 바로 적응한다

셋째, 모든 결과가 기존 모델들을 상회한다

Conclusion

Contributions

- **이질성 해결의 새로운 패러다임 (Soft Prompt):**
  - 기존에는 로봇마다 액션 헤드를 따로 만들거나 전체 모델을 재학습해야 했지만, X-VLA는 하드웨어별 Soft Prompt를 도입해 단 1%의 파라미터 튜닝만으로도 로봇 간 이질성 흡수

- **효율성과 확장성 (Scalability & Efficiency):**
  - 0.9B라는 상대적으로 가벼운 모델로 수십억 파라미터 모델을 능가하는 SOTA 성능 달성

- **실제 로봇 환경의 검증:**
  - 단순 시뮬레이션에 그치지 않고, 3개의 실제 로봇(WidowX, Agilex, AIRBOT)에서 정교한 조작(옷 개기 등)을 성공시켜 현장 적용 가능성 입증

Conclusion

Limitations

- **절대적 규모의 제한:**
  0.9B 파라미터는 로봇 분야에선 크지만, 현재의 거대 언어 모델(LLM)과 비교하면 여전히 작다. LLM이 보여준 '규모의 법칙(Scaling Law)'이 로봇 지능에서도 그대로 성립할지는 아직 미지수

- **수동적 적응의 필요성 (Plug-and-Play의 과제):**
  - 여전히 새로운 로봇에 적응하기 위해 소량의 데모 데이터(Few-shot)가 필요하다. 진짜 '플러그 앤 플레이'는 단순히 기계를 연결하면 끝나는 것인데, 현재는 여전히 하드웨어별 적응 과정(Prompt 학습)이 수반되어야 함

- **Sparse한 지도 신호의 한계:**
  - "왜" 그 행동을 해야 하는지(의도, 다단계 추론)에 대한 고수준의 이해는 부족

- **하드웨어 및 시뮬레이션 환경의 한계:**
  - 아직 세상에 존재하는 모든 로봇의 데이터를 다루기엔 부족하며, 더 다양하고 광범위한 로봇 플랫폼과 데이터셋 필요